



MSC ACADEMY UNIT TEST

Week 9

Dario Di Palma

COS'È UNO UNIT TEST

Un **Unit Test** è un test automatizzato che verifica il corretto funzionamento di **una singola unità di codice**, di solito una funzione o un metodo.

Obiettivo

Garantire che il **comportamento del codice sia corretto** in modo isolato.

Favorire la **manutenibilità**, la **sicurezza nelle modifiche** e la **documentazione viva** del comportamento atteso.

Caratteristiche ideali (i “FIRST principles”)

Fast → devono essere veloci.

Isolated / Independent → non devono dipendere da altri test o risorse esterne.

Repeatable → devono produrre lo stesso risultato ogni volta.

Self-validating → devono fallire o passare automaticamente, senza interpretazione umana.

Timely → dovrebbero essere scritti **insieme o prima del codice** (TDD).

PERCHÈ SCRIVERE UNIT TEST

Benefici

- **Riduzione dei bug** e regressioni.
- **Facilitano il refactoring** (puoi modificare il codice con più sicurezza).
- **Documentano** il comportamento del codice.
- **Favoriscono design migliore** (codice più modulare, meno accoppiato).
- **Abilitano Continuous Integration / Continuous Delivery (CI/CD).**

Limiti

- Non sostituiscono gli altri tipi di test.
- Non garantiscono che il sistema nel suo complesso funzioni (testano solo unità).
- Scrivere test scadenti può rallentare anziché aiutare.

STRUTTURA DI UN UNIT TEST

Pattern “AAA”

1. **Arrange** → prepara il contesto e i dati.
2. **Act** → esegui il codice da testare.
3. **Assert** → verifica che il risultato sia quello atteso.

Esempio (C# con xUnit):

```
[Fact]
public void Sum_ShouldReturnCorrectResult()
{
    // Arrange
    var calc = new Calculator();

    // Act
    var result = calc.Sum(2, 3);

    // Assert
    Assert.Equal(5, result);
}
```

XUNIT

xUnit è un **framework di unit testing** usato per scrivere test automatizzati in C# che utilizza gli attributi per marcare i metodi come Unit Test.

Attributo [Fact]

```
[Fact]
public void Sum_ShouldReturnCorrectResult()
{
    // Arrange
    var calculator = new Calculator();

    // Act
    var result = calculator.Sum(2, 3);

    // Assert
    Assert.Equal(5, result);
}
```

- [Fact] indica che il metodo è un unit test.
- Assert.Equal(expected, actual) verifica che il risultato sia quello atteso.

Attributo [Theory]

```
[Theory]
[InlineData(2, 3, 5)]
[InlineData(-1, 1, 0)]
[InlineData(0, 0, 0)]
public void Sum_ShouldHandleVariousCases(int a, int b, int expected)
{
    var calculator = new Calculator();
    var result = calculator.Sum(a, b);
    Assert.Equal(expected, result);
}
```

- [Theory] indica che il test verrà eseguito più volte con input diversi.
- [InlineData(...)] fornisce i valori di input per ogni esecuzione.

ISOLAMENTO E DIPENDENZE

Unità isolata

L'unit test deve testare **solo la logica interna** dell'unità.

Se la classe usa:

- Database
- File system
- API esterne
→ bisogna **simulare (mockare)** quelle dipendenze.

Tecniche di isolamento

- **Stub** → ritorna risposte predefinite.
- **Mock** → verifica che una dipendenza sia stata chiamata in un certo modo.
- **Fake** → implementazione semplice di un componente reale.

Librerie comuni: *Moq, NSubstitute, FakeItEasy*

MOQ

Moq è una libreria .NET per il **mocking**, cioè per creare **oggetti finti (mock)** durante gli unit test.

Serve ad **isolare il codice che si sta testando** dalle sue dipendenze esterne, come database, servizi web, file system o altre classi.

Funzionamento

- **Creazione di un mock** dell'interfaccia da sostituire.
- **Definire il comportamento** delle chiamate ai metodi del mock (es. cosa deve restituire).
- **Verificare le interazioni** con il mock (es. se un metodo è stato chiamato una volta, con certi parametri).

BEST PRACTICE

- Scrivere test **semplici e leggibili**
- Un solo “concetto” per test (un comportamento, non più di uno)
- Nomina chiara: `MethodName_StateUnderTest_ExpectedBehavior`
- Evitare test dipendenti tra loro
- Evitare l’uso di logica complessa nei test (no if, for, ecc.)
- Tenere i test vicino al codice (/Tests folder o progetto parallelo)
- Automatizzare l’esecuzione (CI pipeline)

CODE COVERAGE

La **code coverage** è una **metrica** che indica **quale percentuale del codice sorgente viene eseguita** durante l'esecuzione dei test automatici.

- **Verifica la copertura dei test:** mostra se i test toccano tutte le parti critiche del codice.
- **Aiuta a trovare codice non testato o “zombie”** (mai eseguito).
- **Misura indirettamente la qualità del testing**, anche se non garantisce che i test siano buoni.
- **Supporta il refactoring:** se hai test con buona coverage, puoi modificare il codice con più sicurezza.
- **Permette di impostare metriche minime di qualità**, ad esempio “almeno 80% di coverage” nei processi CI/CD.

ERRORI COMUNI

- Test troppo complessi o fragili
- Mock eccessivi (test che non riflettono la realtà)
- Scrivere test dopo il codice solo per aumentare la Code Coverage
- Non aggiornare i test quando cambia la logica
- Usare test per validare comportamento non deterministico (es. tempo, random)

INTEGRATION TEST

Un **Integration Test** (o test di integrazione) è un tipo di test automatico che **verifica l'interazione tra più componenti del sistema** — ad esempio, tra moduli di codice, servizi, database o API.

Mentre un **Unit Test** testa una singola unità di codice (tipicamente un metodo o una classe) in isolamento, un **Integration Test** controlla che **le unità funzionino correttamente insieme**.

Lo scopo degli Integration Test è assicurarsi che:

- le diverse parti del sistema **collaborino correttamente**
- le **dipendenze esterne** (database, API, filesystem, ecc.) funzionino come previsto
- il comportamento complessivo sia **coerente e affidabile** anche in un contesto “reale”.

In breve:

- Gli Unit Test garantiscono che “ogni mattoncino funzioni”.
- Gli Integration Test garantiscono che “i mattoncini funzionino correttamente insieme”.

TEST DRIVEN DEVELOPMENT

Il **Test Driven Development (TDD)** è una **metodologia di sviluppo del software** che mette i **test al centro del processo di programmazione**.

L'idea di base è: **scrivere prima i test**, e solo dopo scrivere il codice necessario per farli passare.

Vantaggi del TDD

- **Qualità del codice più alta:** i test scritti prima costringono a progettare codice modulare, testabile e con dipendenze chiare.
- **Feedback immediato:** se qualcosa non funziona come previsto, i test lo segnalano subito.
- **Meno bug in produzione:** ogni parte del codice è verificata da test automatici.
- **Facilità nel refactoring:** i test agiscono come rete di sicurezza durante le modifiche.
- **Documentazione:** i test descrivono il comportamento atteso del codice in modo esplicito.

TDD: RED – GREEN - REFACTOR

1. RED – Scrivi un test che fallisce

- Prima di scrivere qualsiasi logica applicativa, si crea un test automatico che descrive il comportamento desiderato.
- Poiché il codice che deve essere testato non esiste ancora, il test fallisce (rosso).
- Questo passaggio costringe lo sviluppatore a **pensare al comportamento e ai requisiti prima del codice**.

2. GREEN – Scrivi il codice minimo per far passare il test

- Si scrive solo la quantità minima di codice necessaria per far sì che il test passi (verde).
- Non si pensa all'ottimizzazione o alla pulizia del codice in questa fase.
- L'obiettivo è ottenere un **risultato corretto e verificato automaticamente**.

3. REFACTOR – Migliora il codice mantenendo i test verdi

- Ora che il test è verde, si può rifattorizzare (pulire) il codice:
 - migliorare i nomi delle variabili e dei metodi;
 - eliminare duplicazioni;
 - semplificare la logica;
 - migliorare la struttura complessiva.
- Tutti i test devono continuare a passare: questo garantisce che la rifattorizzazione non abbia introdotto regressioni.

